

UNIVERZA NA PRIMORSKEM
FAKULTETA ZA MATEMATIKO, NARAVOSLOVJE IN
INFORMACIJSKE TEHNOLOGIJE

Zaključna naloga

(Final project paper)

**Modeliranje in razumevanje sodobnih sistemov za zaznavanje
omrežnih vdorov**

(Modeling and Understanding of Modern Network Intrusion Detection Systems)

Ime in priimek: Zhivko Stoimchev

Študijski program: Računalništvo in informatika

Mentor: doc. dr. Aleksandar Tošić, PhD

Somentor: asist. Domen Vake

Koper, avgust 2026

Ključna dokumentacijska informacija

Ime in PRIIMEK: Zhivko STOIMCHEV

Naslov zaključne naloge: Modeliranje in razumevanje sodobnih sistemov za zaznavanje omrežnih vdorov

Kraj: Koper

Leto: 2026

Število listov: 31

Število slik: 4

Število tabel: 10

Število referenc: 24

Mentor: doc. dr. Aleksandar Tošić, PhD

Somentor: asist. Domen Vake

Ključne besede: zaznavanje omrežnih vdorov, napadi DDoS, strojno učenje, globoko učenje, omrežni tokovi, vrednotenje modelov

Izvleček: Raziskave sistemov za zaznavanje omrežnih vdorov pogosto poročajo o natančnosti blizu 99 %, vendar na rezultate vplivajo razlike v podatkovnih zbirkah, značilkah, postopkih predobdelave in načinih delitve podatkov. Razvit je bil enoten eksperimentalni postopek za primerjavo osnovnega modela večinskega razreda, modela XGBoost, večplastnega perceptrona, modela z enotami GRU in modela na osnovi arhitekture Transformer pod skupnimi pogoji.

Modeli so bili ovrednoteni na podatkih o omrežnih tokovih CIC z naključno delitvijo ter izločitvami družine in podtipov napadov. Uporabljena je bila tudi zunanja podatkovna zbirka botnetnega prometa. Pri naključni delitvi je XGBoost dosegel najvišjo mero F1, in sicer 0,9553, Transformer pa je bil najboljši nevronskega modela z mero F1 0,9329. Ko je bila iz učnih podatkov izločena celotna družina napadov DDoS, se je najvišja mera F1 zmanjšala na 0,5865. Razvrstitev modelov se je razlikovala tudi med izločitvama LOIC-HTTP in HOIC, uporaba vseh 57 značilk pa ni prinesla doslednega izboljšanja. Največji padec je nastal pri zunanjem vrednotenju, kjer večina modelov ni zaznala nobenega od 400.000 zlonamernih zapisov. Rezultati kažejo, da visoka uspešnost pri naključni delitvi ne zagotavlja zanesljivega zaznavanja nevidenih napadov ali prometa iz drugega omrežnega okolja.

Key words documentation

Name and SURNAME: Zhivko STOIMCHEV

Title of final project paper: Modeling and Understanding of Modern Network Intrusion Detection Systems

Place: Koper

Year: 2026

Number of pages: 31

Number of figures: 4

Number of tables: 10

Number of references: 24

Mentor: doc. dr. Aleksandar Tošić, PhD

Co-Mentor: asist. Domen Vake

Keywords: network intrusion detection, DDoS attacks, machine learning, deep learning, network flows, model evaluation

Abstract: Network intrusion detection studies often report accuracy close to 99%, but their results are affected by differences in datasets, features, preprocessing procedures, and data-splitting strategies. A unified experimental pipeline was developed to compare a majority-class baseline, XGBoost, a multilayer perceptron, a gated recurrent unit model, and a Transformer-style model under common conditions.

The models were evaluated using CIC network-flow data with random, attack-family, and attack-subtype holdouts, together with an external botnet dataset. On the random split, XGBoost achieved the highest F1-score of 0.9553, while the Transformer was the strongest neural model with 0.9329. When the complete DDoS family was excluded from training, the highest F1-score decreased to 0.5865. Model rankings also differed between the LOIC-HTTP and HOIC holdouts, and using all 57 CIC features did not consistently improve performance.

External evaluation produced the largest decrease. Most models detected none of the 400,000 malicious botnet records. The results show that high performance under random splitting does not guarantee reliable recognition of unseen attacks or traffic from another network environment.

Acknowledgement

I would like to express my sincere gratitude to my mentor, doc. dr. Aleksandar Tošić, and my co-mentor, asist. Domen Vake, for their guidance, valuable feedback, and support throughout the preparation of this final project paper. Their advice helped me improve both the technical work and the presentation of the research.

I am deeply grateful to my family and my girlfriend for always standing by my side, believing in me, and encouraging me during the more demanding periods of my studies. I would also like to thank my close friends for their support, understanding, and companionship throughout my student journey.

Their encouragement and confidence in me made this achievement possible.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem statement	2
1.3	Aim and Objectives	2
2	Review of Previous Work	4
2.1	Intrusion Detection Datasets	4
2.2	Intrusion Detection Approaches	5
2.2.1	Classical Machine-Learning Approaches	5
2.2.2	Feedforward Neural Networks	5
2.2.3	Recurrent Neural Networks and GRU Models	6
2.3	Previous Systematic and Experimental Work	6
2.4	Limitations of Existing Evaluations	7
3	Methodology	8
3.1	Experimental Design	8
3.2	Datasets	8
3.3	Data Preprocessing	9
3.3.1	Label Normalisation	9
3.3.2	Data Cleaning	9
3.3.3	Feature Sets	9
3.4	Data Splits	10
3.5	Model Selection	10
3.6	Training Strategy	11
3.7	Evaluation Procedure	11
4	System Design and Implementation	13
4.1	System Architecture	13
4.2	Experiment Configuration	13
4.3	Shared Processing Components	14
4.4	Implemented Models	14

4.4.1	Majority-Class Baseline	14
4.4.2	XGBoost	15
4.4.3	Multilayer Perceptron	15
4.4.4	Gated Recurrent Unit Model	15
4.4.5	Transformer-Style Model	15
4.5	Implementation Validation	16
5	Experiments and Results	17
5.1	Experimental Environment	17
5.2	Random-Split Results	17
5.3	Attack-Holdout Results	18
5.3.1	Complete DDoS-Family Holdout	18
5.3.2	DDoS-Subtype Holdouts	18
5.4	Influence of the Feature Set	20
5.5	Additional GRU Evaluations	21
5.6	External Evaluation	22
5.7	Execution Time	22
5.8	Summary and Limitations	23
6	Conclusion	25
7	Povzetek naloge v slovenskem jeziku	27
8	Bibliography	29

List of Tables

1	Datasets used in the evaluation	9
2	Prepared evaluation configurations	10
3	Results on the random 80/20 split	17
4	Results for the complete DDoS-family holdout	18
5	Results for the LOIC-HTTP subtype holdout	19
6	Results for the HOIC subtype holdout	19
7	LOIC-HTTP F1-score with the common and complete feature sets . . .	20
8	Additional GRU results on the LOIC-HTTP holdout	21
9	Results on the external dataset	22
10	Training time on the HOIC holdout	23

List of Figures

1	Confusion matrix for the GRU on the LOIC-HTTP holdout	19
2	F1-score across the random, family-holdout, subtype-holdout, and external evaluation conditions	20
3	Comparison of the common and complete feature sets on the LOIC-HTTP holdout	21
4	Model training time on the HOIC holdout	23

List of Abbreviations

CIC	Canadian Institute for Cybersecurity
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DDoS	Distributed Denial of Service
DNN	Deep Neural Network
DoS	Denial of Service
GRU	Gated Recurrent Unit
HOIC	High Orbit Ion Cannon
HTTP	Hypertext Transfer Protocol
IDS	Intrusion Detection System
KDD	Knowledge Discovery in Databases
LOIC	Low Orbit Ion Cannon
LSTM	Long Short-Term Memory
MLP	Multilayer Perceptron
NIDS	Network Intrusion Detection System
RNN	Recurrent Neural Network
SVM	Support Vector Machine
TCP	Transmission Control Protocol
VPS	Virtual Private Server

1 Introduction

1.1 Background and Motivation

Computer networks support communication, business operations, public services, and many other activities. Their increasing importance also makes disruption, unauthorised access, and data theft a serious security concern.

A denial-of-service attack attempts to make a system or a service unavailable for use by exhausting its resources. When the traffic originates from multiple devices, it is referred to as a distributed denial-of-service, or DDoS attack. Such attacks affect application servers, websites, banking networks, and other critical network infrastructures.

Network intrusion detection systems monitor the network activity and identify potentially malicious behaviour. Traditional approaches commonly rely on signatures or manually defined rules. However, machine-learning based approaches tend to learn patterns from labelled examples of benign and malicious traffic. Network communication is usually represented through measurable flow characteristics, including duration, transferred bytes, packet counts, traffic rates, transport-layer flags, and other related properties.

Classical machine-learning models, feedforward neural networks, recurrent models, and other deep-learning architectures have all been applied to intrusion detection, and their reported accuracy frequently approaches 99% under the selected benchmark conditions [1, 2, 4, 6]. Such results depend on the dataset used, features selected, preprocessing procedure, class distribution, and method used to divide the data.

However, high performance on a familiar test partition does not necessarily indicate reliable detection of an unseen attack type or traffic originating from another network environment.

1.2 Problem statement

Different intrusion detection studies report different results, and they are difficult to compare directly. They use different datasets, feature representations, labels, preprocessing procedures, model configurations, and evaluation metrics. When one study reports high accuracy, that does not imply that it is a reliably more efficient detector.

Particularly important is the train-test split. With random splitting, closely related flows from the same attack or traffic scenario may be present in both partitions. The test data may consequently be very similar to the records already observed during training, producing an optimistic estimate of performance.

Class imbalance can also create an additional problem. When benign traffic forms the majority class, high accuracy can be achieved while many malicious flows remain undetected. Therefore, accuracy must be interpreted with precision, recall, F1-score, balanced accuracy, and a confusion matrix.

The comparison may be affected by feature availability. A larger feature set provides more information, but it may contain dataset-specific relationships that do not transfer to other attack types or traffic sources. Therefore, a reliable evaluation requires consistent preprocessing, feature definitions, split conditions, and metrics.

1.3 Aim and Objectives

The aim is to compare representative intrusion detection approaches under common evaluation conditions.

Five approaches are included: a majority-class baseline, XGBoost, a multilayer perceptron, a gated recurrent unit model, and a Transformer-style model. All are used for binary classification of benign and malicious network flows.

A CIC-based collection is used as the primary data source. It contains flow records originating from modern CIC intrusion detection datasets, including CIC-IDS 2017, CSE-CIC-IDS 2018, and CIC-DDoS 2019 [19–21]. The datasets provide flow-level statistics and include several intrusion scenarios.

NSL-KDD was considered during the literature review, but was not included in the final implementation. Its 41 connection-level features follow a different representation from the flow statistics used by the CIC datasets [22]. Combining both datasets would require substantial feature transformation and would prevent the use of one consistent input representation. The CIC family was also more suitable for the selected DDoS-

focused models and shared eleven compatible features with the external evaluation data.

The main comparison uses eleven common flow features. A separate experiment uses all 57 available numerical CIC features. The evaluation conditions include several split configurations, including a random split, a complete DDoS-family holdout, LOIC-HTTP and HOIC subtype holdouts, a feature-set comparison, and evaluation on an external dataset containing server-captured benign traffic and injected botnet records.

The analysis examines whether strong random-split performance remains stable when complete attack groups are excluded from training, whether additional features improve generalisation, and whether models trained on the CIC collection transfer to a different traffic distribution.

The goal is a controlled comparison rather than the introduction of a new detection architecture.

2 Review of Previous Work

2.1 Intrusion Detection Datasets

Publicly available datasets are commonly used to develop and compare intrusion detection methods. They contain examples of benign and malicious network activity, but differ in their traffic sources, attack scenarios, feature representations, and class distributions.

KDD Cup 1999 is one of the earliest and most widely used intrusion detection datasets [23]. It represents each network connection through 41 manually engineered features and includes DoS, probing, remote-to-local, and user-to-root attacks. Its main limitation is the age of the traffic and the large number of repeated records.

NSL-KDD was introduced as a revised version of KDD Cup 1999 [22]. It retains the same 41-feature representation, while reducing redundant records and modifying the sample distribution [22]. Although it remains widely used today, its connection-level and host-based features originate from an older and largely artificial network environment.

More recent datasets were produced by the Canadian Institute for Cybersecurity. CIC-IDS-2017 contains benign traffic and several attacks, including DDoS, brute-force, botnet, infiltration, and web attacks [19]. CSE-CIC-IDS-2018 follows a similar flow-based representation and contains traffic collected across several attack scenarios [20]. CIC-DDoS-2019 focuses specifically on DDoS traffic, and includes several flooding, reflection, and amplification attacks [21].

The KDD and CIC dataset families represent traffic differently. KDD-based datasets contain manually engineered connection and host-level features, while CIC datasets contain mainly packet, byte, timing, and TCP-flag statistics extracted from bidirectional flows. Previous compatibility analysis found little meaningful feature overlap between these families, making direct model transfer unsuitable without substantial transformation [18].

2.2 Intrusion Detection Approaches

Both classical machine-learning algorithms and neural-network architectures have been applied to intrusion detection. Their reported performance is strongly influenced by the selected datasets, classification tasks, features, and evaluation procedure.

2.2.1 Classical Machine-Learning Approaches

Halladay et al. compared Random Forest, k-nearest neighbours, LightGBM, XGBoost, AdaBoost, linear discriminant analysis, Gaussian Naive Bayes, support vector machines, and a deep neural network on CIC-DDoS-2019 [2]. Binary classification was performed using both the complete cleaned feature set and a reduced set of time-related features.

Most models achieved accuracy close to 99% in the binary experiment using the complete feature set. XGBoost achieved approximately 99.98% accuracy. Performance decreased with the reduced feature set and in the multiclass task, where the best accuracy was approximately 74%. The large difference shows that binary attack detection is considerably easier than distinguishing individual DDoS types.

Nkima et al. evaluated feature selection on the NSL-KDD dataset using univariate selection, recursive feature elimination, and decision trees [9]. Several reduced subsets performed similarly to, or even better, than the complete 41-feature representation. This indicates that additional features do not necessarily improve classification.

Therefore, tree-based methods are strong candidates for structured flow data, but their performance depends on the information available in the selected features, and on the difficulty of the classification task.

2.2.2 Feedforward Neural Networks

Basnet et al. evaluated feedforward neural networks on CSE-CIC-IDS-2018 using fast.ai and Keras implementations [1]. Models were trained separately on daily traffic files and on combined binary and multiclass datasets.

Most daily experiments achieved accuracy above 98%, and one reached 100%. However, performance on the traffic from 1 March 2018 decreased to approximately 87% with fast.ai and 72% with Keras. The results show that performance can change substantially when the traffic composition changes, even within the same dataset.

Vinayakumar et al. compared multilayer perceptrons and deep belief networks on KDD

Cup 1999 and NSL-KDD [8]. Their results also varied according to network depth and feature selection. Moukafih et al. used several lightweight feedforward networks, whose predictions were combined through a voting mechanism [16].

Feedforward networks provide a useful neural baseline because they process structured numerical features without the additional assumptions introduced by recurrent or convolutional architectures.

2.2.3 Recurrent Neural Networks and GRU Models

Xu et al. proposed an intrusion detection architecture combining gated recurrent units, a multilayer perceptron, and a softmax classifier [4]. The strongest bidirectional GRU configuration achieved 99.84% accuracy on KDD Cup 1999, and 99.24% on NSL-KDD.

Assis et al. evaluated a GRU-based system using CIC-DDoS-2019 and CSE-CIC-IDS2018 [6]. The GRU was compared with DNN, CNN, LSTM, support vector machine, logistic regression, k-nearest neighbours, and other approaches. The reported average performance was approximately 99.94% on CIC-DDoS-2019, while the accuracy on CSE-CIC-IDS2018 was approximately 97.10%.

The difference between the two datasets shows that the same architecture may produce different results when the traffic and attack scenarios change. Similar behaviour was observed in CNN-RNN, CNN-LSTM, and CNN-GRU combinations evaluated by Vinayakumar [7].

Recurrent architectures are normally intended for sequential data. However, in several intrusion detection implementations, the features of one record are treated as sequence positions rather than processing consecutive network flows. Such models should not automatically be interpreted as learning temporal behaviour between communications.

2.3 Previous Systematic and Experimental Work

An earlier systematic analysis by Danilo reviewed intrusion detection papers, the datasets used, reported results, and associated public repositories [18]. The methods were grouped into classical machine learning, feedforward networks, recurrent architectures, clustering, fuzzy systems, and other anomaly-detection approaches.

Particular attention was given to dataset compatibility. KDD-based datasets were found to use connection-level, login-related, and host-window features, while CIC-based datasets used flow-level packet, byte, timing, and TCP-flag statistics. Direct transfer

between the two families was therefore considered inappropriate without extensive feature transformation.

The analysis also investigated a combined CIC collection and reduced feature sets suitable for external evaluation. Existing implementations were audited as references, but direct reproduction of every repository was considered unnecessary because many used similar models under different preprocessing and evaluation conditions.

Traffic captured from a VPS was also examined. Initial behavioural labelling based on flow rates and TCP flags produced unreliable classifications, particularly for failed connections. Since the captured traffic contained very few confirmed malicious records, botnet traffic was later injected in order to provide an external malicious class.

These observations identified feature incompatibility, class imbalance, uncertain labels, and dependence on dataset-specific patterns as important limitations of intrusion detection evaluations.

2.4 Limitations of Existing Evaluations

Accuracy close to 99% appears frequently in the reviewed studies, but it is not consistent across all conditions. Basnet et al. reported almost perfect results on several CIC-IDS-2018 traffic days, but considerably weaker results on another day [1]. Halladay et al. obtained near-perfect binary results with the complete CIC-DDoS-2019 features, while multiclass and reduced-feature results were lower [2]. Assis et al. also reported different GRU performance on CIC-DDoS-2019 and CIC-IDS-2018 datasets [6].

Most studies trained and tested their models on traffic originating from the same dataset. Random splitting may place closely related flows from the same attack scenario in both partitions, making the test set similar to the training data.

Class distribution further affects interpretation. A high accuracy value may mainly reflect correct classification of the majority class, while many attacks remain undetected. Balanced subsets simplify model training but may not represent the traffic distribution of an operational network.

The use of different feature sets and preprocessing procedures also limits direct comparison. Cross-dataset testing is uncommon and is often restricted by incompatible feature representations. The earlier systematic analysis similarly found that very high native-dataset performance did not necessarily transfer to external traffic [18].

Therefore, reported results should be understood as performance under specific datasets and evaluation conditions.

3 Methodology

3.1 Experimental Design

The evaluation is formulated as a binary classification task. Each network flow is assigned either to the benign class or to the malicious class.

Five approaches are compared: a majority-class baseline, XGBoost, a multilayer perceptron, a gated recurrent unit model, and a Transformer-style model. They represent different levels of complexity, ranging from a trivial predictor to classical machine learning and neural-network architectures.

The main evaluation uses a common CIC-based dataset, shared feature definitions, prepared data splits, and the same performance metrics. In addition to a conventional random split, the complete DDoS family and individual DDoS subtypes are excluded from training. A separate external dataset is used to examine performance under a different traffic distribution.

Six configurations are included: a random split, a complete DDoS-family holdout, LOIC-HTTP and HOIC subtype holdouts, a LOIC-HTTP holdout using the complete feature set, and an external evaluation.

3.2 Datasets

The primary dataset is a combined CIC collection containing flow records from several CIC intrusion detection datasets [18–21]. It contains 9,167,581 records, of which 7,186,189 are benign and 1,981,392 are malicious. The malicious records include DDoS, DoS, botnet, brute-force, infiltration, web attack, and port-scanning traffic.

The external dataset contains traffic captured from a VPS environment and 400,000 known botnet records added as a malicious class [18]. It contains 1,687,049 flows in total. It is used only for external evaluation and not for model training. Its purpose is to measure cross-dataset robustness, rather than represent a controlled live DDoS environment.

Table 1: Datasets used in the evaluation

Dataset	Total records	Benign	Malicious
CIC collection	9,167,581	7,186,189	1,981,392
External dataset	1,687,049	1,287,049	400,000

3.3 Data Preprocessing

3.3.1 Label Normalisation

The original CIC labels describe individual attack types. These labels are converted into a common binary representation. Benign traffic is assigned the value 0, while every attack category is assigned the value 1.

The original attack name is retained only when required to construct an attack-family or subtype holdout. It is removed before model training and is never used as an input feature.

3.3.2 Data Cleaning

Column names are converted to lowercase, spaces and special characters are replaced with underscores, and known alternative names are mapped to a common representation.

Identifiers and metadata that could reveal the origin of a record are excluded from the model inputs. These include flow identifiers, source and destination addresses, timestamps, capture names, and dataset-source fields.

Selected features are converted to numerical values. Infinite values are replaced with missing values, and records containing invalid values in the final feature set are removed. The prepared files therefore contain only numerical input features and one binary label column.

3.3.3 Feature Sets

The main feature set contains eleven numerical fields available in both the CIC collection and the external dataset. This allows the same trained models to be evaluated without changing their expected inputs.

A second CIC-only feature set contains all 57 usable numerical fields. It includes additional packet-length, inter-arrival-time, header, subflow, active, and idle statistics. The

complete representation is used only for the LOIC-HTTP feature comparison because these additional fields are not available in the external dataset.

3.4 Data Splits

The random configuration uses a stratified 80/20 split with seed 42. Stratification preserves approximately the same benign and malicious proportions in both partitions.

The holdout configurations remove a complete attack group from the training set. The training set contains 80% of the benign records and all malicious records except the held-out group. The test set contains the remaining 20% of benign records and every record belonging to the held-out attack group.

The complete DDoS-family holdout excludes all records classified as DDoS. The subtype configurations exclude only DDoS-LOIC-HTTP or DDoS-HOIC, while other attack categories remain available during the training.

Table 2: Prepared evaluation configurations

Configuration	Training	Testing	Held-out traffic
Random 80/20	7,334,064	1,833,517	No held-out attack group
Complete DDoS-family holdout	6,495,614	2,671,967	All DDoS records
LOIC-HTTP subtype holdout	7,154,979	2,012,602	DDoS-LOIC-HTTP
HOIC subtype holdout	7,531,482	1,636,099	DDoS-HOIC
LOIC-HTTP full-feature holdout	7,154,979	2,012,602	DDoS-LOIC-HTTP
External evaluation	–	1,687,049	Complete external dataset

The external configuration contains no training partition. Models trained on the CIC random split are loaded and evaluated directly on the complete external dataset.

3.5 Model Selection

The majority-class baseline provides a reference for interpreting accuracy under class imbalance. It always predicts the most frequent training class.

XGBoost represents classical machine learning and was selected because tree-based boosting methods achieved strong results in the reviewed DDoS research [2]. The multilayer perceptron provides a simple neural baseline based on the feedforward architectures used in previous intrusion detection work [1].

The GRU represents recurrent neural networks and follows the general model family evaluated on the CIC datasets by Assis et al. [6]. Its inputs are the feature values of one flow arranged as sequence positions: it does not process consecutive flows over time.

The Transformer-style model is included as a modern extension. It represents each numerical feature as a token and applies attention between the feature representations.

3.6 Training Strategy

All models used the complete prepared training partition. The majority-class baseline and XGBoost used all records directly. For the neural models, 90% of the partition was used for parameter fitting and 10% was reserved for validation.

The numerical features used by the neural models were standardised using the mean and standard deviation, calculated from the complete prepared training partition. The same values were later applied to the validation and test records. Test records were not used during training or preprocessing.

Neural-model training used binary cross-entropy with logits, the AdamW optimiser, a batch size of 4096, a learning rate of 0.001, weight decay of 0.0001, and ten epochs. The default decision threshold was 0.5. Class weighting and automatic threshold selection were disabled in the final configurations.

Additional GRU evaluations examined validation-based threshold selection and positive-class weighting. Neither produced better held-out test performance, so the default threshold and unweighted loss were retained.

The external evaluation reused the model artifacts trained on the complete CIC random-split training partition with seed 42. The expected feature names, feature order, and neural-model standardisation values were restored from each saved artifact before prediction.

3.7 Evaluation Procedure

Malicious traffic is treated as the positive class. The reported measures are accuracy, balanced accuracy, precision, recall, F1-score, macro F1-score, weighted F1-score, and the confusion matrix.

Accuracy measures the overall proportion of correct predictions. Balanced accuracy

gives equal importance to the benign and the malicious traffic. Precision describes how many predicted attacks are actually malicious, while recall describes how many malicious records are detected. The F1-score combines precision and recall and is used as the principal single-value measure.

The confusion matrix records true negatives, false positives, false negatives, and true positives. It distinguishes false alarms from undetected attacks, which cannot be understood from accuracy alone.

Training, evaluation, and total execution times are also recorded. Conclusions are based on performance across all configurations, rather than on the highest result from one split.

4 System Design and Implementation

4.1 System Architecture

The software was implemented in Python 3.11 as a modular command-line application. It provides a common structure for dataset preparation, model training, model evaluation, and result analysis.

The main entry point is the `run.py` file, which forwards the selected operation to an experiment orchestrator. Data processing is performed using `pandas`, `NumPy`, and `PyArrow`. `Scikit-learn` and `XGBoost` are used for classical machine learning, while the neural models are implemented using `PyTorch`. `Matplotlib` is used for creating the final figures.

The implementation is divided into configuration registries, shared processing components, and model-specific modules. Dataset preparation and evaluation are handled centrally, while each model contains only its architecture and model-specific training logic. This avoids unnecessary duplication and ensures that all models follow the same experimental procedure.

The complete implementation and experiment configurations are available in the accompanying source-code repository [24].

4.2 Experiment Configuration

Datasets, feature sets, data splits, and models are defined in separate JSON registries. The dataset registry contains the data location, format, label column, and fields excluded from training. The feature registry defines the common 11-feature representation and the complete numerical CIC representation. The split registry describes how each training and testing partition is constructed, while the model registry defines the available model modules.

The command-line interface supports dataset inspection, split preparation, model training, model evaluation, result summarisation, and figure generation. Each execution specifies the model, data split and random seed.

External evaluation also requires the original training split to be specified. This ensures that the model selected for external testing was trained using the correct CIC configuration.

4.3 Shared Processing Components

A common preparation component normalises column names and labels, selects the required features, converts them to numerical values, and removes invalid records. It also constructs the random, attack-family, subtype-holdout, and external evaluation partitions described in Chapter 3.

The neural models use a shared PyTorch training procedure. Numerical features are standardised using values calculated from the training partition, after which 10% of the data is reserved for validation. The same loss function, optimiser, batch size, learning rate, number of epochs, and decision threshold are used for the MLP, GRU, and Transformer-style models.

Evaluation is also handled centrally. The expected feature order and neural-model standardisation values are restored before prediction. The same metric component calculates accuracy, balanced accuracy, precision, recall, F1-scores, and confusion matrices for every model.

4.4 Implemented Models

4.4.1 Majority-Class Baseline

The majority-class baseline determines the most frequent class in the training partition and assigns that class to every test record.

The baseline does not learn any traffic patterns. Its purpose is to verify the operation of the pipeline and provide a reference for interpreting accuracy on imbalanced datasets.

4.4.2 XGBoost

The XGBoost implementation uses a binary gradient-boosted tree classifier. It contains 200 trees with a maximum depth of six and a learning rate of 0.1. Row and feature subsampling are both set to 0.8. Histogram-based tree construction is also used.

The complete numerical training partition is passed directly to the classifier. Feature standardisation is not required because tree-based models do not depend on the numerical scale of individual features.

4.4.3 Multilayer Perceptron

The multilayer perceptron receives one numerical feature vector for each network flow. It contains two fully connected hidden layers with 64 and 32 neurons.

Each hidden layer is followed by a rectified linear unit activation and dropout with a probability of 0.1. The final layer produces the binary classification score.

4.4.4 Gated Recurrent Unit Model

The GRU model represents the feature values of one flow as an ordered sequence. Each sequence position contains one numerical value, producing an input with a feature dimension of one.

The model contains one unidirectional GRU layer with a hidden size of 64. The final hidden state is processed through layer normalisation, dropout with a probability of 0.1, and a binary classification layer.

The sequence represents the features of one flow rather than consecutive flows in time. The model therefore learns relationships between ordered feature values and does not model temporal communication between separate network flows.

4.4.5 Transformer-Style Model

The Transformer-style model represents each numerical feature as an individual token. Every scalar value is projected into a 32-dimensional vector, and a learned feature embedding is added so that the positions can be distinguished.

The tokens are processed by two Transformer encoder layers containing four attention heads. Each encoder uses a feedforward dimension of 128, GELU activation, and dropout with a probability of 0.1.

The encoded tokens are averaged into one flow representation, which is passed through a small classification network to produce the binary output.

4.5 Implementation Validation

All five models were successfully trained and evaluated using the final configurations. The implementation recorded the selected dataset, feature set, split, model, seed, execution time, and evaluation metrics for each run.

The final results summary was generated from an explicit selection of completed runs. This ensured that the tables and figures were based on the intended full-training configurations, rather than earlier development runs.

5 Experiments and Results

5.1 Experimental Environment

All experiments were executed on a Linux system, running an Intel Core i7-1260P processor, and 16 GB of memory. Neural-model training was performed on the CPU using Python 3.11 and PyTorch. The same prepared partitions, random seed, and model evaluation procedure were used for all models within each configuration.

The final comparison contains five models and six evaluation configurations. The main results use the common 11-feature representation, while the separate LOIC-HTTP feature comparison uses all 57 numerical CIC features.

5.2 Random-Split Results

Table 3 presents the results obtained using the stratified 80/20 random split.

Table 3: Results on the random 80/20 split					
Model	Accuracy	Balanced accuracy	Precision	Recall	F1-score
Baseline	0.7839	0.5000	0.0000	0.0000	0.0000
XGBoost	0.9811	0.9634	0.9797	0.9321	0.9553
MLP	0.9645	0.9416	0.9322	0.9012	0.9165
GRU	0.9673	0.9489	0.9310	0.9165	0.9237
Transformer	0.9713	0.9541	0.9423	0.9237	0.9329

XGBoost achieved the strongest overall results, with an F1-score of 0.9553 and balanced accuracy of 0.9634. The Transformer-style model was the strongest neural approach, followed by the GRU and MLP.

The majority-class baseline achieved an accuracy of 0.7839 because benign traffic formed the larger class. However, it detected no malicious records, resulting in zero recall and an F1-score of zero. This demonstrates why accuracy alone is insufficient for imbalanced intrusion detection data.

All learned models performed strongly under random splitting. However, related records from the same traffic and attack scenarios may appear in both partitions, so these values provide a favourable benchmark, rather than sufficient evidence of generalisation.

5.3 Attack-Holdout Results

5.3.1 Complete DDoS-Family Holdout

The complete DDoS-family holdout excludes every DDoS record from training. The models are trained using benign traffic and other malicious categories and are then evaluated on benign traffic and previously unseen DDoS records.

Table 4: Results for the complete DDoS-family holdout

Model	Accuracy	Balanced accuracy	Precision	Recall	F1-score
Baseline	0.5379	0.5000	0.0000	0.0000	0.0000
XGBoost	0.7284	0.7064	0.9890	0.4169	0.5865
MLP	0.6868	0.6620	0.9633	0.3349	0.4970
GRU	0.6517	0.6234	0.9858	0.2498	0.3986
Transformer	0.6592	0.6318	0.9749	0.2695	0.4222

Performance decreased considerably compared with the random split. XGBoost remained the strongest model, but its F1-score fell from 0.9553 to 0.5865. Its precision remained very high at 0.9890, while recall decreased to 0.4169.

Therefore, the models produced relatively few false alarms, but failed to detect a large proportion of the unseen DDoS flows. Learning other intrusion categories was not sufficient for reliable recognition of an entirely excluded attack family.

5.3.2 DDoS-Subtype Holdouts

The subtype experiments exclude only LOIC-HTTP or HOIC from training. Other DDoS and intrusion categories remain available, allowing the models to learn broader malicious traffic characteristics.

The GRU model produced the strongest LOIC-HTTP results, achieving an F1-score of 0.9017 and recall of 0.9115. It correctly detected 524,468 of the 575,364 held-out LOIC-HTTP flows. The remaining models showed considerably lower recall.

The HOIC results followed a different ranking.

Table 5: Results for the LOIC-HTTP subtype holdout

Model	Accuracy	Balanced accuracy	Precision	Recall	F1-score
Baseline	0.7141	0.5000	0.0000	0.0000	0.0000
XGBoost	0.8256	0.6990	0.9685	0.4032	0.5693
MLP	0.8350	0.7244	0.9147	0.4662	0.6176
GRU	0.9432	0.9337	0.8920	0.9115	0.9017
Transformer	0.8012	0.6820	0.8029	0.4037	0.5372

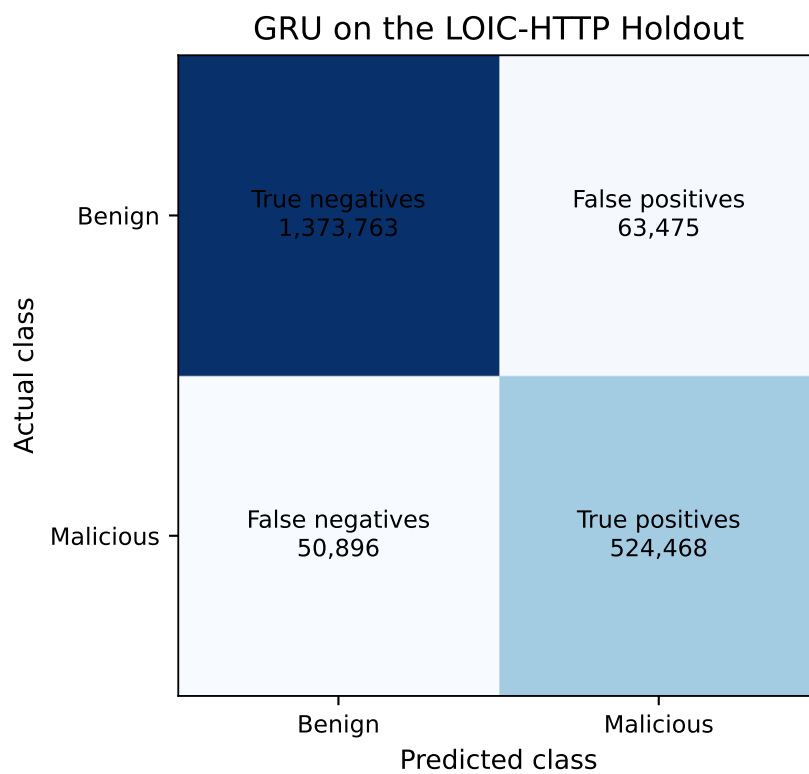


Figure 1: Confusion matrix for the GRU on the LOIC-HTTP holdout

Table 6: Results for the HOIC subtype holdout

Model	Accuracy	Balanced accuracy	Precision	Recall	F1-score
Baseline	0.8785	0.5000	0.0000	0.0000	0.0000
XGBoost	0.9897	0.9749	0.9600	0.9554	0.9577
MLP	0.9667	0.9215	0.8640	0.8618	0.8629
GRU	0.9571	0.8882	0.8414	0.7972	0.8187
Transformer	0.9760	0.9671	0.8619	0.9553	0.9062

XGBoost achieved the strongest HOIC result, with an F1-score of 0.9577. The Transformer also achieved high recall and an F1-score of 0.9062. In contrast to the LOIC-HTTP result, the GRU ranked last among the learned models.

The difference between the two subtype experiments shows that no architecture was consistently the strongest. The GRU generalised particularly well to LOIC-HTTP, while XGBoost was considerably more effective for HOIC.

Figure 2 summarises the F1-scores across the main evaluation conditions.

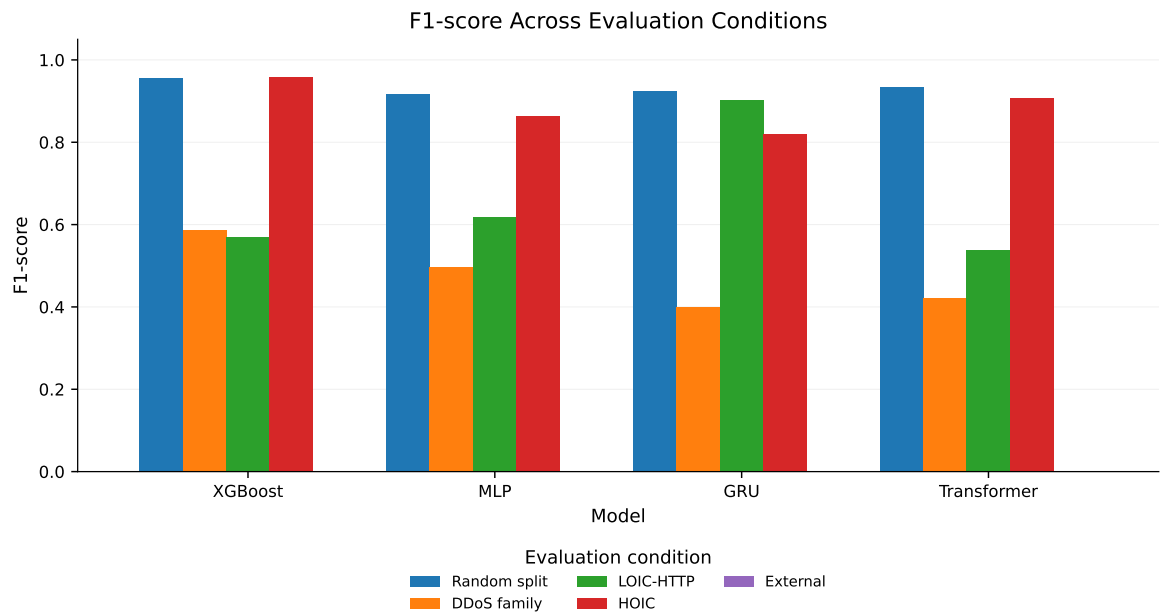


Figure 2: F1-score across the random, family-holdout, subtype-holdout, and external evaluation conditions

5.4 Influence of the Feature Set

The LOIC-HTTP experiment was repeated using all 57 numerical CIC features. The training and testing records remained unchanged, allowing the effect of the feature representation to be examined separately.

Table 7: LOIC-HTTP F1-score with the common and complete feature sets

Model	11 common features	57 complete features
XGBoost	0.5693	0.0028
MLP	0.6176	0.5985
GRU	0.9017	0.6280
Transformer	0.5372	0.5880

The complete feature set did not provide a consistent improvement. The Transformer improved moderately, but the MLP and GRU produced lower F1-scores. The largest difference occurred for XGBoost, whose F1-score decreased from 0.5693 to 0.0028.

The additional fields may contain relationships that are useful for distinguishing traffic already represented during training, but do not transfer to the excluded LOIC-HTTP subtype. Therefore, more input features did not necessarily provide better generalisation.

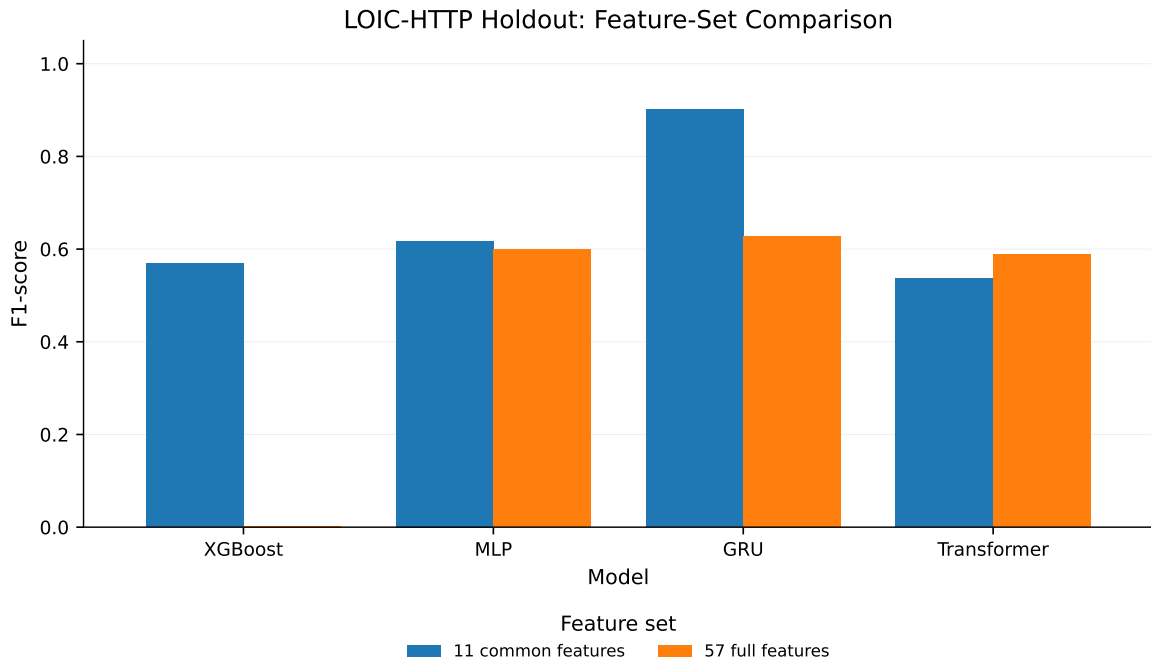


Figure 3: Comparison of the common and complete feature sets on the LOIC-HTTP holdout

5.5 Additional GRU Evaluations

Validation-based threshold selection and positive-class weighting were examined for the GRU model on the LOIC-HTTP holdout.

Table 8: Additional GRU results on the LOIC-HTTP holdout

Configuration	Threshold	Test F1-score
Default configuration	0.50	0.9017
Validation-selected threshold	0.75	0.8705
Positive-class weighting	0.50	0.5698

The threshold of 0.75 improved validation F1-score from 0.8654 to 0.9014, but reduced

the held-out test F1-score to 0.8705. Positive-class weighting produced a further decrease to 0.5698. The standard threshold and unweighted loss were therefore retained.

5.6 External Evaluation

The models trained on the CIC random split were evaluated directly on the external dataset. The external malicious class contained 400,000 injected botnet flows and therefore represents a different traffic and attack distribution from the CIC training data.

Table 9: Results on the external dataset

Model	Accuracy	Balanced accuracy	True positives	False positives
Baseline	0.7629	0.5000	0	0
XGBoost	0.7626	0.4998	0	481
MLP	0.7629	0.5000	12	18
GRU	0.2363	0.1549	0	888,387
Transformer	0.7626	0.4998	0	437

The baseline, XGBoost, GRU, and Transformer detected none of the 400,000 malicious records. The MLP detected only 12, producing an attack recall of 0.00003.

Accuracy remained close to 0.763 for most models because benign traffic formed approximately three quarters of the external dataset. These values therefore mainly reflect majority-class behaviour rather than successful intrusion detection.

The GRU behaved differently, predicting 888,387 benign records as malicious while still failing to detect any botnet record. This produced balanced accuracy of only 0.1549 and indicates a severe mismatch between the learned CIC decision boundary and the external feature distribution.

The external results demonstrate that identical feature names and model inputs do not guarantee compatible traffic distributions. Strong performance on the CIC collection did not transfer to the external botnet data.

5.7 Execution Time

Execution time was compared using the HOIC configuration because complete timing information was available for all models.

XGBoost achieved the strongest HOIC result while requiring considerably less training

Table 10: Training time on the HOIC holdout

Model	Training time
Baseline	3 seconds
XGBoost	2 minutes 7 seconds
MLP	15 minutes 53 seconds
GRU	33 minutes 57 seconds
Transformer	2 hours 13 minutes 38 seconds

time than the neural models. The Transformer was the most computationally expensive model and required more than two hours of CPU training.

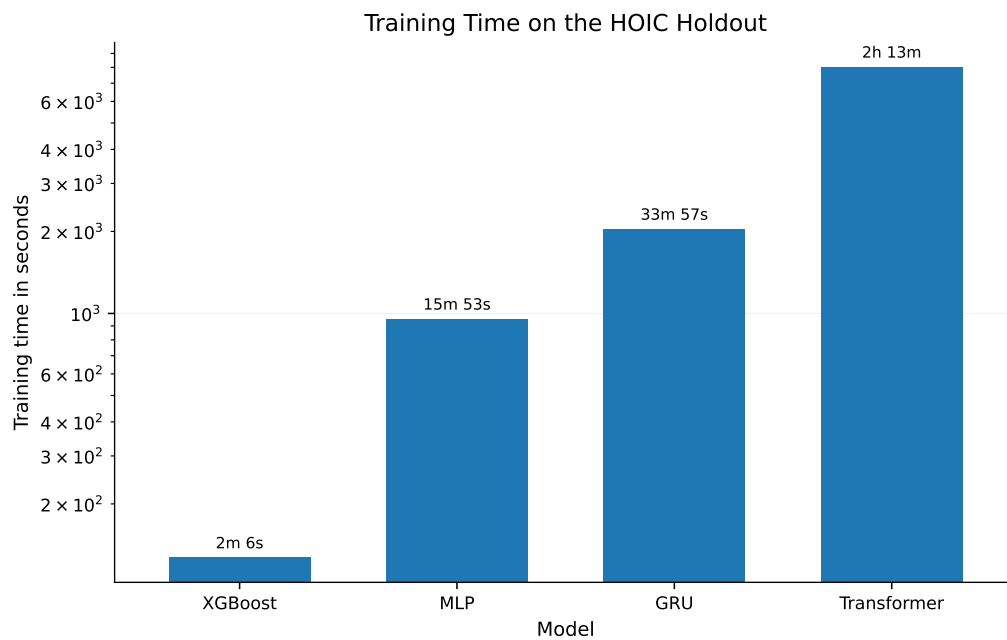


Figure 4: Model training time on the HOIC holdout

5.8 Summary and Limitations

The random split produced strong results for every learned model, with XGBoost achieving the highest F1-score. Performance decreased when the complete DDoS family was excluded from training. The subtype experiments were less consistent: the GRU was the strongest for LOIC-HTTP, while XGBoost was the strongest for HOIC.

Using all 57 CIC features did not provide a consistent advantage over the reduced feature representation. The external evaluation produced the largest performance decrease, with almost no malicious flows detected by any model.

The results are limited to binary flow classification and one random seed. The GRU processes the features of an individual flow as a sequence and does not analyse consecutive flows over time. The available CIC data also did not preserve sufficient provenance for a fully temporal or source-based split. Finally, the external malicious class consists of injected botnet traffic rather than confirmed live DDoS attacks.

Despite these limitations, the results show that performance depends strongly on the evaluation condition. High performance under random splitting did not imply reliable recognition of excluded attack groups or traffic from another dataset.

6 Conclusion

A common experimental pipeline was developed for comparing five network intrusion detection approaches: a majority-class baseline, XGBoost, a multilayer perceptron, a gated recurrent unit model, and a Transformer-style model. The same datasets, feature definitions, data splits, training procedure, and evaluation metrics were used to reduce differences unrelated to the models themselves.

The random 80/20 split produced strong results for all learned models. XGBoost achieved the highest F1-score of 0.9553, while the Transformer-style model was the strongest neural approach with an F1-score of 0.9329. These results confirmed that the selected architectures can classify CIC network flows effectively when training and testing records originate from the same collection.

Performance changed considerably under more restrictive evaluation conditions. When the complete DDoS family was excluded from training, the highest F1-score decreased to 0.5865. The subtype experiments produced different model rankings. The GRU achieved the strongest LOIC-HTTP result with an F1-score of 0.9017, while XGBoost achieved the strongest HOIC result with an F1-score of 0.9577. Therefore, no model was consistently superior for every unseen attack category.

Using the complete 57-feature representation did not consistently improve generalisation. The MLP and GRU produced weaker LOIC-HTTP results, while XGBoost almost completely failed to recognise the held-out subtype. Additional features may provide useful information within a familiar distribution, but they may also introduce dataset-specific relationships that do not transfer to unseen traffic.

The largest performance decrease occurred during external evaluation. XGBoost, the GRU, and the Transformer detected none of the 400,000 malicious botnet records, while the MLP detected only 12. Most accuracy values remained close to the proportion of benign traffic and therefore did not represent successful attack detection. The GRU additionally produced a very large number of false positives, showing a severe difference between the CIC and external feature distributions.

The reported high results from existing research should not be interpreted as incorrect. They describe performance under particular datasets, features, class distributions, and

splitting strategies. The completed evaluations show that high random-split performance does not guarantee reliable detection of excluded attack groups or traffic from another network environment.

The comparison was limited to binary flow classification, one random seed, and datasets with incomplete temporal and source provenance. The external malicious class also consisted of injected botnet records rather than confirmed live DDoS attacks. Future work should therefore use traffic with preserved timestamps and network identifiers, repeated evaluations with several seeds, multiclass attack labels, and destination-based temporal aggregation. Evaluation on independently collected and reliably labelled operational traffic would provide a stronger assessment of practical intrusion detection performance.

7 Povzetek naloge v slovenskem jeziku

Sistemi za zaznavanje omrežnih vdorov spremljajo omrežni promet in poskušajo prepoznati zlonamerno dejavnost. Pri tem se vse pogosteje uporabljajo modeli strojnega in globokega učenja, ki se učijo iz označenih primerov legitimnega in zlonamerne prometa. Raziskave na tem področju pogosto poročajo o natančnosti blizu ali nad 99 odstotki, vendar so rezultati pridobljeni z uporabo različnih podatkovnih zbirk, značilk, postopkov predobdelave in načinov delitve podatkov. Zato rezultatov posameznih modelov ni mogoče neposredno primerjati, visoka natančnost pa ne pomeni nujno uspešnega zaznavanja nove vrste napada ali prometa iz drugega omrežnega okolja.

Namen naloge je bil primerjati več reprezentativnih pristopov za zaznavanje omrežnih vdorov pod enakimi pogoji. Primerjani so bili osnovni model večinskega razreda, XG-Boost, večplastni perceptron, model z enotami GRU in model na osnovi arhitekture Transformer. Vsi modeli so bili uporabljeni za binarno klasifikacijo omrežnih tokov na legitimne in zlonamerne.

Kot glavni vir podatkov je bila uporabljena združena zbirka CIC z 9.167.581 omrežnimi tokovi. Od tega je bilo 7.186.189 legitimnih in 1.981.392 zlonamernih zapisov. Glavna primerjava je uporabljala enajst značilk, ki opisujejo trajanje toka, število paketov in bajtov, povprečne dolžine paketov, hitrost prometa ter zastavice TCP. Ločena primerjava je uporabila vseh 57 razpoložljivih številskih značilk.

Pripravljenih je bilo šest konfiguracij vrednotenja. Poleg običajne naključne delitve podatkov v razmerju 80 proti 20 je bila iz učnih podatkov izločena celotna družina napadov DDoS. Ločeno sta bila izločena tudi podtipa LOIC-HTTP in HOIC. Poskus z napadom LOIC-HTTP je bil ponovljen z vsemi 57 značilkami. Za preverjanje prenosa modelov na drugačno porazdelitev prometa je bila uporabljena še zunanja zbirka, ki vsebuje legitimni promet, zajet v okolju navideznega zasebnega strežnika, in 400.000 dodanih zapisov znanega botnetnega prometa.

Pri naključni delitvi so vsi naučeni modeli dosegli visoke rezultate. Najboljši je bil XG-Boost z mero F1 0,9553. Med nevronskimi modeli je bil najboljši Transformer z mero

F1 0,9329, sledila sta GRU z 0,9237 in večplastni perceptron z 0,9165. Osnovni model je dosegel natančnost 0,7839, vendar ni zaznal nobenega zlonamernega zapisa. To kaže, da sama natančnost pri neuravnoteženih podatkih ni zadostna mera uspešnosti.

Pri izločitvi celotne družine napadov DDoS se je uspešnost izrazito zmanjšala. Najboljši rezultat je ponovno dosegel XGBoost, vendar je njegova mera F1 padla na 0,5865. Modeli so ohranili visoko preciznost, vendar so spregledali velik delež prej nevidenih napadov.

Rezultati obeh podtipov napadov so pokazali drugačno razvrstitev modelov. Pri izločitvi LOIC-HTTP je bil najboljši GRU z mero F1 0,9017, medtem ko je bil pri izločitvi HOIC najboljši XGBoost z mero F1 0,9577. Transformer je pri napadu HOIC dosegel mero F1 0,9062. Različni rezultati kažejo, da nobena arhitektura ni bila najboljša pri vseh vrstah napadov.

Uporaba vseh 57 značilk ni prinesla splošnega izboljšanja. Pri napadu LOIC-HTTP se je uspešnost večine modelov zmanjšala, pri XGBoostu pa je mera F1 padla na 0,0028. Večje število značilk lahko vsebuje povezave, ki so koristne znotraj znane podatkovne porazdelitve, vendar se ne prenesejo nujno na neviden tip napada.

Največji padec uspešnosti je nastal pri zunanjem vrednotenju. XGBoost, GRU in Transformer niso zaznali nobenega od 400.000 zlonamernih zapisov, večplastni perceptron pa jih je zaznal samo 12. Visoka natančnost večine modelov je bila predvsem posledica večjega deleža legitimnega prometa. GRU je poleg tega napačno označil 888.387 legitimnih tokov kot zlonamerne. Rezultati kažejo, da enaka imena značilk ne zagotavljajo enake porazdelitve podatkov in da modeli, naučeni na zbirki CIC, niso uspešno prenesli svojega delovanja na zunanji promet.

Rezultati obstoječih raziskav zato niso nujno napačni, vendar opisujejo uspešnost v določenih eksperimentalnih pogojih. Visoka uspešnost pri naključni delitvi ni zagotavljala zanesljivega zaznavanja izločenih napadov ali prometa iz drugega okolja.

Glavne omejitve so binarna klasifikacija, uporaba enega naključnega semena, pomanjkanje popolnih časovnih in izvornih podatkov ter uporaba dodanega botnetnega prometa namesto potrjenih aktivnih napadov DDoS. Nadaljnje delo bi lahko vključilo večkratno vrednotenje z različnimi semeni, večrazredno klasifikacijo, časovno združevanje omrežnih tokov ter preverjanje na neodvisno zbranem in zanesljivo označenem operativnem prometu.

8 Bibliography

- [1] R. B. BASNET, R. SHASH, C. JOHNSON, L. WALGREN, and T. DOLECK, Towards Detecting and Classifying Network Intrusion Traffic Using Deep Learning Frameworks. *Journal of Internet Services and Information Security* 9 (2019) 1–17. *(Cited on pages 1, 5, 7, and 10.)*
- [2] J. HALLADAY, D. CULLEN, N. BRINER, J. WARREN, K. FYE, R. BASNET, J. BERGEN, and T. DOLECK, Detection and Characterization of DDoS Attacks Using Time-Based Features. *IEEE Access* 10 (2022) 49794–49807. *(Cited on pages 1, 5, 7, and 10.)*
- [3] S. AGARWAL, A. TYAGI, and G. USHA, A Deep Neural Network Strategy to Distinguish and Avoid Cyber-Attacks. In: S. S. Dash, C. Lakshmi, S. Das, B. K. Panigrahi (eds.), *Artificial Intelligence and Evolutionary Computations in Engineering Systems*, Springer, Singapore, 2020, 673–681. *(Not cited.)*
- [4] C. XU, J. SHEN, X. DU, and F. ZHANG, An Intrusion Detection System Using a Deep Neural Network With Gated Recurrent Units. *IEEE Access* 6 (2018) 48697–48707. *(Cited on pages 1 and 6.)*
- [5] Q. NIYAZ, W. SUN, A. Y. JAVAID, and M. ALAM, A Deep Learning Approach for Network Intrusion Detection System. In *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies*, 2016, 21–26. *(Not cited.)*
- [6] M. V. O. ASSIS, L. F. CARVALHO, J. LLORET, and M. L. PROENÇA JR., A GRU deep learning system against attacks in software defined networks. *Journal of Network and Computer Applications* 177 (2021) 102942. *(Cited on pages 1, 6, 7, and 11.)*
- [7] R. VINAYAKUMAR, K. P. SOMAN, and P. POORNACHANDRAN, Applying Convolutional Neural Network for Network Intrusion Detection. In *2017 International Conference on Advances in Computing, Communications and Informatics*, 2017, 1222–1228. *(Cited on page 6.)*

- [8] R. VINAYAKUMAR, K. P. SOMAN, and P. POORNACHANDRAN, Evaluating Effectiveness of Shallow and Deep Networks to Intrusion Detection System. In *2017 International Conference on Advances in Computing, Communications and Informatics*, 2017, 1282–1289. (Cited on page 6.)
- [9] H. NKIAMA, S. Z. MOHD SAID, and M. SAIDU, A Subset Feature Elimination Mechanism for Intrusion Detection System. *International Journal of Advanced Computer Science and Applications* 7 (2016) 148–157. (Cited on page 5.)
- [10] J. PEREZ and B. DANNER, Optimizing preprocessing and parameters for cluster-based anomaly detection. In *Research Experiences for Undergraduates, Montana State University*, 2019, Research poster. (Not cited.)
- [11] B. M. SERINELLI, A. COLLEN, and N. A. NIJDAM, On the Analysis of Open Source Datasets: Validating IDS Implementation for Well-Known and Zero-Day Attack Detection. *Procedia Computer Science* 191(2021) 192–199. (Not cited.)
- [12] B. M. SERINELLI, A. COLLEN, and N. A. NIJDAM, Training Guidance with KDD Cup 1999 and NSL-KDD Data Sets of ANIDINR: Anomaly-Based Network Intrusion Detection System. *Procedia Computer Science* 175 (2020) 560–565. (Not cited.)
- [13] F. H. BOTES, L. LEENEN, and R. DE LA HARPE, Ant Colony Induced Decision Trees for Intrusion Detection. In *Proceedings of the 16th European Conference on Cyber Warfare and Security*, 2017, 53–62. (Not cited.)
- [14] J. E. DICKERSON, J. JUSLIN, O. KOUKOUSOULA, and J. A. DICKERSON, Fuzzy Intrusion Detection. In *Proceedings of the Joint 9th IFSA World Congress and 20th NAFIPS International Conference*, 2001, 1506–1510. (Not cited.)
- [15] J. E. DICKERSON and J. A. DICKERSON, Fuzzy Network Profiling for Intrusion Detection. In *Proceedings of the 19th International Conference of the North American Fuzzy Information Processing Society*, 2000, 301–306. (Not cited.)
- [16] N. MOUKAFIH, G. ORHANOU, and S. EL HAJJI, Neural network-based voting system with high capacity and low computation for intrusion detection in SIEM/IDS systems. *Security and Communication Networks* 2020 (2020) Article ID 3512737, 15 pages. (Cited on page 6.)
- [17] S. ZHAI, Y. CHENG, W. LU, and Z. ZHANG, Deep Structured Energy Based Models for Anomaly Detection. In *Proceedings of the 33rd International Conference on Machine Learning*, 2016, 1100–1109. (Not cited.)

- [18] DANILO LAZIĆ, *Evaluating Transferability of Machine Learning Models for DDoS Detection*. Unpublished undergraduate research manuscript, University of Primorska, Faculty of Mathematics, Natural Sciences and Information Technologies, 2026. (*Cited on pages 4, 6, 7, and 8.*)
- [19] *CIC-IDS-2017*, Canadian Institute for Cybersecurity, University of New Brunswick. <https://www.unb.ca/cic/datasets/ids-2017.html>. (Viewed on: 26/7/2026.) (*Cited on pages 2, 4, and 8.*)
- [20] *CSE-CIC-IDS2018*, Canadian Institute for Cybersecurity, University of New Brunswick. <https://www.unb.ca/cic/datasets/ids-2018.html>. (Viewed on: 26/7/2026.) (*Cited on pages 2, 4, and 8.*)
- [21] *CIC-DDoS2019*, Canadian Institute for Cybersecurity, University of New Brunswick. <https://www.unb.ca/cic/datasets/ddos-2019.html>. (Viewed on: 26/7/2026.) (*Cited on pages 2, 4, and 8.*)
- [22] M. TAVALLAEE, E. BAGHERI, W. LU, and A. A. GHORBANI, A Detailed Analysis of the KDD CUP 99 Data Set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009, 1–6. (*Cited on pages 2 and 4.*)
- [23] S. STOLFO, W. FAN, W. LEE, A. PRODROMIDIS, and P. CHAN, *KDD Cup 1999*, <https://archive.ics.uci.edu/dataset/130/kdd+cup+1999+data>. (Viewed on: 26/7/2026.) (*Cited on page 4.*)
- [24] Z. STOIMCHEV, *Source code for Modeling and Understanding of Modern Network Intrusion Detection Systems*, <https://github.com/zstoimchev/thesis26>. (Viewed on: 26/7/2026.) (*Cited on page 13.*)